Nama: Muhammad Fadhil Ilyas
NIM: 434241099
Kelas: TI-C6
Link Github: https://github.com/ilysthecreator/PBL_Praktikum.git

Modul 12: Testing & Quality, API Documentation, dan Docker Deployment (Edisi Puzzle — Analisis dan Perbaikan Kesalahan Sistem Pengujian, Dokumentasi OpenAPI, dan Kontainerisasi)

D.1 Berita Acara Perbaikan (Tabel 9 Kesalahan)

Modul 12 Edisi Puzzle memuat 9 kesalahan yang sengaja ditanam pada kode sumber awal. Berdasarkan panduan 'Bagian B.1 — Berburu Kesalahan pada Kode Sumber', kesalahan dibagi ke dalam tiga kelompok: kesalahan kompilasi (compiler errors), kesalahan unit test & isolasi memory, serta kesalahan pengujian integrasi database & konkurensi. Seluruh kesalahan telah berhasil diidentifikasi, dibuktikan gejalanya sebelum perbaikan, dan diperbaiki hingga seluruh spesifikasi penerimaan terpenuhi.

No	Lokasi (File, Fungsi)	Gejala yang Diamati	Pemeriksaan yang Mengungkap	Akar Masalah	Perbaikan
1	app/service/review_rules_test.go TestNormalizeTags	Gagal kompilasi: cannot use t (variable of type testing.T) as *testing.T in argument to t.Helper / t.Errorf	compiler	Parameter test dideklarasikan sebagai value type (t testing.T) alih-alih pointer (*testing.T). Method pengujian Go memerlukan pointer receiver.	Mengubah deklarasi parameter menjadi pointer: func TestNormalizeTags(t *testing.T).
2	app/service/fakes_test.go fakeReviewRepo.FindByCourse	Gagal kompilasi: cannot use &fakeReviewRepo{} as repository.ReviewRepository (missing method FindByCourse)	compiler	Signature method pada fakeReviewRepo menerima courseID int, sedangkan interface repository.ReviewRepository membutuhkan q model.ReviewQuery.	Menyesuaikan signature method fake menjadi FindByCourse(ctx context.Context, q model.ReviewQuery) ([]model.Review, error) dan menyaring data sesuai query.
3	app/service/review_errors_test.go TestTranslateReviewError	Gagal kompilasi: cannot refer to unexported name service.translateReviewError	compiler	File test berada di package service_test (black-box), namun memanggil fungsi internal tidak terekspos (unexported) translateReviewError.	Mengubah deklarasi package file menjadi package service (white-box test) agar fungsi internal dapat diakses tanpa diekspos keluar.
4	app/service/review_service_test.go newTestEnv	Test gagal acak saat dijalankan dengan pengacakan urutan: TestListReviews_NewCourseIsEmpty mendapati data ulasan dari test lain.	T2 (go test -shuffle=on -count=5)	Menggunakan global variable sharedReviews yang dipakai bersama antar test, melanggar prinsip test isolation.	Membuat instansi fake baru di dalam fungsi helper: reviews := newFakeReviewRepo() untuk setiap test case.
5	app/service/notification_worker_test.go TestHandleSend_PermanentFailures	Test gagal dengan pesan kesalahan: err != asynq.SkipRetry bernilai true padahal error merupakan hasil pembungkusan SkipRetry.	T1 (go test -count=1 ./app/service/)	Membandingkan error wrapped secara langsung dengan operator !=, bukan menggunakan errors.Is untuk unwrapping.	Mengganti perbandingan pointer dengan fungsi standar unwrapping: !errors.Is(err, asynq.SkipRetry).
6	app/service/course_r	Uji mutasi T5 tetap	T5 (Mutation	Test suite tidak memiliki	Menambahkan skenario

	ules_test.go TestWithinCreditLimit	hijau (lolos) ketika operator <= diubah menjadi <, padahal logika batas berubah.	<= ke <)	test case untuk nilai batas tepat (boundary value) pada 24 SKS.	uji tepat di batas 24 SKS (taken: 21, adding: 3, want: true) sehingga mutasi langsung tertangkap.
7	tests/integration/main_test.go TestMain	Seluruh integration test dilewati (0 tests run) meskipun database test PostgreSQL telah tersedia dan variabel DSN terisi.	T4 (go test -tags=integration)	Logika pemeriksaan DSN terbalik: if dsn != "" { return } menyebabkan runner keluar prematur saat DSN ditemukan.	Memperbaiki kondisi menjadi if dsn == "" { return } sehingga test suite hanya berhenti jika DSN benar-benar kosong.
8	tests/integration/main_test.go openTestPool	Test konkurensi TestEnrollEndpoint_ConcurrentCreditLimit macet tanpa batas (deadlock / timeout).	T4 (TestEnrollEndpoint_ConcurrentCreditLimit)	Konfigurasi database connection pool untuk pengujian dibatasi hanya 1 koneksi (MaxConns = 1), menimbulkan connection starvation.	Menaikkan batas connection pool pengujian menjadi cfg.MaxConns = 15 agar konkurensi 10 goroutine dapat berjalan paralel.
9	tests/integration/main_test.go seedCourse	Test integrasi gagal dengan status 404 Not Found (kelas tidak ditemukan) segera setelah seedCourse selesai dijalankan.	T4 (TestIncrementEnrolled_StopsAtQuota)	seedCourse menggunakan defer untuk menghapus baris kelas, sehingga data langsung dihapus saat helper fungsi selesai return.	Mengganti defer dengan t.Cleanup(func() { ... }) agar data kelas tetap ada selama test berjalan dan baru dibersihkan setelah test usai.

Rincian dan Bukti Perbaikan Sembilan Kesalahan (Bagian B.1)

1. Kesalahan 1: Parameter testing.T Non-Pointer pada Unit Test Review Rules
Kode sebelum perbaikan:
func TestNormalizeTags(t testing.T) { ... }
Keluaran compiler sebelum perbaikan:

```
# latihan-fiber/app/service_test [latihan-fiber/app/service.test]
app/service/review_rules_test.go:12:15: cannot use t (variable of type testing.T) as *testing.T
in argument to t.Helper
app/service/review_rules_test.go:15:10: cannot use t (variable of type testing.T) as *testing.T
in argument to t.Errorf
FAIL    latihan-fiber/app/service [build failed]
```

Perbaikan kode:
func TestNormalizeTags(t *testing.T) { ... }
Keluaran setelah perbaikan: Kompilasi sukses dan pengujian TestNormalizeTags lulus 100%.

2. Kesalahan 2: Ketidaksesuaian Interface pada fakeReviewRepo
Kode sebelum perbaikan:
func (f *fakeReviewRepo) FindByCourse(ctx context.Context, courseID int) ([]model.Review, error) { ... }
Keluaran compiler sebelum perbaikan:

```
# latihan-fiber/app/service_test [latihan-fiber/app/service.test]
app/service/review_service_test.go:42:25: cannot use &fakeReviewRepo{} (value of type
*fakeReviewRepo) as repository.ReviewRepository in argument to service.NewReviewService:
*fakeReviewRepo does not implement repository.ReviewRepository (wrong type for method
FindByCourse)
        have FindByCourse(context.Context, int) ([]model.Review, error)
        want FindByCourse(context.Context, model.ReviewQuery) ([]model.Review, error)
FAIL    latihan-fiber/app/service [build failed]
```

Perbaikan kode:
func (f *fakeReviewRepo) FindByCourse(ctx context.Context, q model.ReviewQuery) ([]model.Review,

```
error) {
    f.mu.Lock()
    defer f.mu.Unlock()
    var result []model.Review
    for _, r := range f.reviews {
        if r.CourseID == q.CourseID {
            result = append(result, r)
        }
    }
    return result, nil
}
```

Keluaran setelah perbaikan: fakeReviewRepo memenuhi kontrak interface repository.ReviewRepository.

3. Kesalahan 3: Visibilitas Fungsi Internal translateReviewError (Black-box vs White-box)

Kode sebelum perbaikan:

```
package service_test
func TestTranslateReviewError(t *testing.T) {
    err := service.translateReviewError(errors.New("..."))
}
```

Keluaran compiler sebelum perbaikan:

```
# latihan-fiber/app/service_test [latihan-fiber/app/service.test]
app/service/review_errors_test.go:18:17: cannot refer to unexported name
service.translateReviewError
FAIL    latihan-fiber/app/service [build failed]
```

Perbaikan kode:

```
package service
// Dengan menjadi package service, test menjadi white-box dan dapat mengakses translateReviewError
secara langsung tanpa prefix service.
```

Keluaran setelah perbaikan: Seluruh 9 skenario pemetaan translateReviewError teruji dan lulus.

4. Kesalahan 4: Shared Mutable State Antar Test Case (Pelanggaran Test Isolation)

Gejala diamati saat menjalankan perintah T2 (pengacakan shuffle dan repetisi):

```
go test -p 1 -vet=off -buildvcs=false -shuffle=on -count=5 ./app/service/
```

```
--- FAIL: TestListReviews_NewCourseIsEmpty (0.00s)
    review_service_test.go:88: jumlah ulasan = 1, seharusnya 0
FAIL
FAIL    latihan-fiber/app/service   0.142s
```

Akar masalah: Variabel global package var sharedReviews = newFakeReviewRepo() digunakan oleh seluruh fungsi pengujian. Data yang disisipkan oleh TestCreateReview_Success tersisa di memori saat TestListReviews_NewCourseIsEmpty dieksekusi kemudian.

Perbaikan: Mengisolasi repositori fake di dalam helper instansiasi per-test:

```
func newTestEnv(t *testing.T) testEnv {
    t.Helper()
    reviews := newFakeReviewRepo() // fake baru dan bersih untuk setiap test
    courses := &fakeCourseRepo{ ... }
    ...
}
```

Keluaran setelah perbaikan: T2 lulus 100% pada 3 kali repetisi berturut-turut tanpa kebocoran state.

5. Kesalahan 5: Pembandingan Error Langsung terhadap Wrapped Sentinel Error
Kode sebelum perbaikan:

```
if err != asynq.SkipRetry {
   t.Fatalf("seharusnya SkipRetry, dapat: %v", err)
}
```

Keluaran sebelum perbaikan:

```
--- FAIL: TestHandleSend_PermanentFailures (0.00s)
    --- FAIL: TestHandleSend_PermanentFailures/user_sudah_dihapus (0.00s)
        notification_worker_test.go:78: seharusnya SkipRetry, dapat: user tidak ditemukan:
asynq.SkipRetry
FAIL
FAIL    latihan-fiber/app/service   0.089s
```

Perbaikan kode:

```
if !errors.Is(err, asynq.SkipRetry) {
   t.Fatalf("seharusnya SkipRetry, dapat: %v", err)
}
```

Keluaran setelah perbaikan: Pengujian mengenali error yang terbungkus fmt.Errorf("...: %w", asynq.SkipRetry) dan lulus sempurna.

6. Kesalahan 6: Kekosongan Boundary Test Case pada WithinCreditLimit (Terungkap Uji Mutasi T5)
Sebelum perbaikan, fungsi WithinCreditLimit hanya diuji dengan taken=0, 15, 22, 24. Jika operator diubah dari <= menjadi <, pengujian tetap lulus (false green) karena tidak ada data uji pada batas tepat 24 SKS.
Bukti kegagalan uji mutasi T5 sebelum perbaikan ditambahkan (mutasi tidak tertangkap):
Mutasi: taken+adding < MaxCreditsPerSemester -> go test ./app/service/ -> PASS (False Positive!)
Perbaikan: Menambahkan skenario uji tepat di batas 24 SKS:

```
{"tepat di batas 24 SKS", 21, 3, true},
```

Bukti setelah skenario ditambahkan (uji mutasi berhasil menangkap bug):

```
PS C:\Kuliah\Backend\Modul-12> go test -p 1 -vet=off -buildvcs=false -count=1 ./app/service/
--- FAIL: TestWithinCreditLimit (0.00s)
    --- FAIL: TestWithinCreditLimit/tepat_di_batas_24_SKS (0.00s)
        course_rules_test.go:29: WithinCreditLimit(21, 3) = false, seharusnya true
FAIL
FAIL    latihan-fiber/app/service   0.077s
```

Setelah kode dikembalikan ke <=, seluruh test suite kembali hijau dan terbukti tahan terhadap mutasi batas.

7. Kesalahan 7: Logika Terbalik pada TestMain Integration Test
Kode sebelum perbaikan:

```
dsn := os.Getenv("TEST_DATABASE_URL")
if dsn != "" {
   log.Println("TEST_DATABASE_URL kosong, dilewati")
   return
}
```

Keluaran sebelum perbaikan:

```
PS C:\Kuliah\Backend\Modul-12> go test -tags=integration -count=1 -v ./tests/integration/
2026/11/09 13:00:00 TEST_DATABASE_URL kosong, dilewati
testing: warning: no tests to run
PASS
ok      latihan-fiber/tests/integration   0.040s [no tests to run]
```

Akar masalah: Operator != membuat TestMain justru keluar saat variabel koneksi DSN terisi.
Perbaikan kode:

```
if dsn == "" {
   log.Println("TEST_DATABASE_URL kosong, dilewati")
   return
}
```

Keluaran setelah perbaikan: TestMain berhasil menghubungkan pool ke PostgreSQL praktikum_backend_test dan menjalankan seluruh integration test.

8. Kesalahan 8: Connection Pool Starvation pada Test Konkurensi
Kode sebelum perbaikan:

```
cfg.MaxConns = 1
```

Keluaran sebelum perbaikan: TestEnrollEndpoint_ConcurrentCreditLimit macet total (deadlock) karena 10 goroutine berebut 1 koneksi database tunggal hingga runner timeout setelah 10 menit.
Perbaikan kode:

```
cfg.MaxConns = 15
```

Keluaran setelah perbaikan: 10 request konkurensi diproses secara paralel dalam waktu 0.49 detik tanpa hambatan.

9. Kesalahan 9: Premature Deletion pada Helper seedCourse
Kode sebelum perbaikan:

```
func seedCourse(t *testing.T, pool *pgxpool.Pool, code string, credits, quota int) int {
   // ... INSERT INTO courses ...
   defer func() {
      _, _ = pool.Exec(context.Background(), "DELETE FROM courses WHERE id = $1", id)
   }()
   return id
}
```

Keluaran sebelum perbaikan:

```
--- FAIL: TestIncrementEnrolled_StopsAtQuota (0.01s)
    course_repository_test.go:48: kelas tidak ditemukan: no rows in result set
FAIL
FAIL    latihan-fiber/tests/integration   0.215s
```

Akar masalah: Keyword defer membersihkan baris kelas seketika saat seedCourse kembali (return), sehingga tabel courses sudah kosong saat test runner mengeksekusinya.
Perbaikan kode: Menggunakan t.Cleanup agar eksekusi pembersihan ditunda hingga seluruh lifecycle test case selesai:

```
t.Cleanup(func() {
   _, _ = pool.Exec(context.Background(), "DELETE FROM courses WHERE id = $1", id)
})
```

Keluaran setelah perbaikan: Seluruh data seed bertahan selama masa pengujian dan dibersihkan secara rapi setelah test usai.

C. Hasil Pengujian Spesifikasi Penerimaan

C.1 Pengujian Testing (T1 sampai T7)

Berikut adalah bukti keluaran terminal untuk seluruh pengujian spesifikasi penerimaan pengujian (Bagian C.1):

a. Bukti Uji T1 (go test -count=1 ./...): Seluruh package lulus tanpa ada kegagalan.

```
PS C:\Kuliah\Backend\Modul-12> go test -p 1 -vet=off -buildvcs=false -count=1 ./...
?       latihan-fiber                   [no test files]
?       latihan-fiber/app/model         [no test files]
?       latihan-fiber/app/repository    [no test files]
ok      latihan-fiber/app/service       0.106s
?       latihan-fiber/config            [no test files]
?       latihan-fiber/database          [no test files]
?       latihan-fiber/docs              [no test files]
?       latihan-fiber/helper            [no test files]
?       latihan-fiber/middleware        [no test files]
?       latihan-fiber/route             [no test files]
?       latihan-fiber/tools/redeliver   [no test files]
```

b. Bukti Uji T2 (go test -shuffle=on -count=5 ./app/service/): Diuji tiga kali berturut-turut, ketiganya lulus sempurna.

```
PS C:\Kuliah\Backend\Modul-12> for ($i=1; $i -le 3; $i++) { go test -p 1 -vet=off -buildvcs=false
-shuffle=on -count=5 ./app/service/ }
- Pengulangan 1: ok    latihan-fiber/app/service    0.132s
- Pengulangan 2: ok    latihan-fiber/app/service    0.129s
- Pengulangan 3: ok    latihan-fiber/app/service    0.108s
```

c. Bukti Uji T3 (go test -coverprofile=coverage.out & go tool cover -func=coverage.out): Keempat fungsi aturan murni mencapai tepat 100.0%.

```
PS C:\Kuliah\Backend\Modul-12> go tool cover -func=coverage.out | Select-String
"WithinCreditLimit|CreditsAfterSwap|NormalizeTags|BuildMessage"
latihan-fiber/app/service/course_rules.go:13:        WithinCreditLimit       100.0%
latihan-fiber/app/service/course_rules.go:18:        CreditsAfterSwap        100.0%
latihan-fiber/app/service/notification_rules.go:14:  BuildMessage            100.0%
latihan-fiber/app/service/review_rules.go:11:        NormalizeTags           100.0%
```

d. Bukti Uji T4 (go test -tags=integration -count=1 -v ./tests/integration/): Seluruh 8 test tingkat atas lulus (6 test wajib + 2 test mandiri D.2) tanpa FAIL dan tanpa pesan dilewati.

```
PS C:\Kuliah\Backend\Modul-12> go test -tags=integration -count=1 -vet=off -p 1 -buildvcs=false -
v ./tests/integration/
=== RUN   TestIncrementEnrolled_StopsAtQuota
--- PASS: TestIncrementEnrolled_StopsAtQuota (0.01s)
=== RUN   TestInsertEnrollment_Duplicate
--- PASS: TestInsertEnrollment_Duplicate (0.02s)
=== RUN   TestWithTx_RollsBackEveryStep
--- PASS: TestWithTx_RollsBackEveryStep (0.01s)
=== RUN   TestClaim_OnlyOnce
--- PASS: TestClaim_OnlyOnce (0.01s)
=== RUN   TestEnrollEndpoint_WritesOutboxOnlyOnSuccess
--- PASS: TestEnrollEndpoint_WritesOutboxOnlyOnSuccess (0.02s)
=== RUN   TestEnrollEndpoint_ConcurrentCreditLimit
--- PASS: TestEnrollEndpoint_ConcurrentCreditLimit (0.49s)
=== RUN   TestSwapEndpoint_TargetCourseFull
--- PASS: TestSwapEndpoint_TargetCourseFull (0.04s)
=== RUN   TestDropEndpoint_Success
--- PASS: TestDropEndpoint_Success (0.02s)
PASS
ok      latihan-fiber/tests/integration   1.052s
```

e. Bukti Uji Mutasi T5 (course_rules.go: ganti <= menjadi < pada WithinCreditLimit): TestWithinCreditLimit WAJIB gagal.

```
PS C:\Kuliah\Backend\Modul-12> go test -p 1 -vet=off -buildvcs=false -count=1 ./app/service/
--- FAIL: TestWithinCreditLimit (0.00s)
    --- FAIL: TestWithinCreditLimit/tepat_di_batas_24_SKS (0.00s)
        course_rules_test.go:29: WithinCreditLimit(21, 3) = false, seharusnya true
FAIL
FAIL    latihan-fiber/app/service   0.077s
```

f. Bukti Uji Mutasi T6 (course_repository.go: hapus FOR UPDATE pada LockStudent): TestEnrollEndpoint_ConcurrentCreditLimit WAJIB gagal pada setiap pengulangan.

```
PS C:\Kuliah\Backend\Modul-12> go test -tags=integration -count=3 -vet=off -p 1 -buildvcs=false -
run Concurrent ./tests/integration/
--- FAIL: TestEnrollEndpoint_ConcurrentCreditLimit (0.03s)
    enroll_api_test.go:210: berhasil = 10, seharusnya 8 (hasil lengkap: map[201:10])
    enroll_api_test.go:215: baris enrollments = 10, seharusnya 8
--- FAIL: TestEnrollEndpoint_ConcurrentCreditLimit (0.03s)
    enroll_api_test.go:210: berhasil = 10, seharusnya 8 (hasil lengkap: map[201:10])
    enroll_api_test.go:215: baris enrollments = 10, seharusnya 8
--- FAIL: TestEnrollEndpoint_ConcurrentCreditLimit (0.03s)
    enroll_api_test.go:210: berhasil = 10, seharusnya 8 (hasil lengkap: map[201:10])
    enroll_api_test.go:215: baris enrollments = 10, seharusnya 8
FAIL
FAIL    latihan-fiber/tests/integration   0.814s
```

g. Bukti Uji Mutasi T7 (notification_repository.go: hapus ON CONFLICT (outbox_id) DO NOTHING pada Claim): TestClaim_OnlyOnce WAJIB gagal.

```
PS C:\Kuliah\Backend\Modul-12> go test -tags=integration -count=1 -vet=off -p 1 -buildvcs=false -
run Claim ./tests/integration/
--- FAIL: TestClaim_OnlyOnce (0.02s)
    course_repository_test.go:115: claim kedua = false, mencatat notifikasi: ERROR: duplicate key
value violates unique constraint "notifications_outbox_user_unique" (SQLSTATE 23505); seharusnya
false, nil
FAIL
FAIL    latihan-fiber/tests/integration   0.540s
```

C.2 Pengujian Dokumentasi API (OpenAPI Swagger)

Pemeriksaan spesifikasi dokumentasi Swagger menunjukkan hasil sebagai berikut:
1. swag init berhasil dieksekusi tanpa error. Folder docs terisi file docs.go, swagger.json, dan swagger.yaml.
2. File docs/swagger.json memuat 20 path terdaftar (melebihi spesifikasi dasar 11 path modul).
3. Konfigurasi SWAGGER_ENABLED:
   - SWAGGER_ENABLED=true: http://localhost:3000/swagger/index.html menampilkan Swagger UI interaktif dengan judul 'Latihan Fiber Praktikum Backend'.
   - SWAGGER_ENABLED=false: Request ke /swagger/index.html merespons HTTP 404 Not Found.
4. Autentikasi Swagger UI: Menggunakan tombol Authorize dengan skema Bearer <token>. Request GET /courses menghasilkan HTTP 200 OK saat token valid disertakan, dan HTTP 401 Unauthorized bila token tidak dikirim.
5. Skema Data Terstruktur: Pada endpoint POST /courses/{id}/enroll, response 201 mengembalikan skema model.WebResponse dengan field data bertipe Course lengkap (id, code, name, credits, quota, enrolled_count), bukan sekadar tipe generik object.
6. Kompilasi: Seluruh unit test dan build aplikasi tetap lolos setelah integrasi swagger.

```
# Eksekusi swag init
PS C:\Kuliah\Backend\Modul-12> & C:/Users/user/go/bin/swag.exe init -d C:/Kuliah/Backend/Modul-12
```

```
-g main.go -o C:/Kuliah/Backend/Modul-12/docs
Generate swagger docs....
Generate general API Info, search dir:C:/Kuliah/Backend/Modul-12
create docs.go at C:/Kuliah/Backend/Modul-12/docs/docs.go
create swagger.json at C:/Kuliah/Backend/Modul-12/docs/swagger.json
create swagger.yaml at C:/Kuliah/Backend/Modul-12/docs/swagger.yaml

# Verifikasi Jumlah Path di swagger.json
Paths count: 20 endpoints terdaftar
```

C.3 Pengujian Docker Deployment

Pemeriksaan spesifikasi kontainerisasi Docker Deployment membuktikan:
1. docker compose config tervalidasi sukses. Nilai secret dijaga menggunakan sintaks ${VARIABLE:?pesan}. Saat JWT_SECRET dihapus sementara dari .env, perintah menolak berjalan dengan pesan: 'JWT_SECRET wajib diisi di .env'.
2. Multi-stage build pada Dockerfile: Image build menggunakan golang:1.26-alpine untuk mengompilasi binary statis (CGO_ENABLED=0 -ldflags="-s -w"), lalu disalin ke image runtime alpine:3.24 yang sangat ringkas.
3. Container security: Aplikasi dijalankan menggunakan non-root user (USER app, UID app), bukan root.
4. dockerignore: File rahasia (.env, .env.*), repository lokal (.git), file log (logs/, *.log), dan laporan test (coverage.out) dicegah masuk ke dalam image Docker.
5. Arsitektur 5 Service: api (port 8080:3000), worker (-role=worker), postgres (postgres:16-alpine), mongo (mongo:9.0), dan redis (redis:7.4-alpine dengan flag --appendonly yes).
6. Isolasi Jaringan: Port database (5432, 27017, 6379) tidak dipublikasikan ke host, mencegah konflik dengan database lokal mahasiswa.
7. Healthcheck & Dependensi: Container api dan worker hanya akan menyala setelah service database berstatus healthy.
8. Persistensi Data: Volume Docker terkelola (pgdata, mongodata, redisdata) menjamin seluruh akun, kelas, dan ulasan tetap aman saat container dimatikan via docker compose down.

```
# Cuplikan Validasi docker compose config
PS C:\Kuliah\Backend\Modul-12> docker compose config
name: modul-12
services:
  api:
    build: { context: ., dockerfile: Dockerfile }
    image: latihan-fiber:local
    command: ["-role=api"]
    ports: ["8080:3000"]
    depends_on:
      postgres: { condition: service_healthy }
      mongo:    { condition: service_healthy }
      redis:    { condition: service_healthy }
  worker:
    image: latihan-fiber:local
    command: ["-role=worker"]
  postgres:
    image: postgres:16-alpine
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./migrations:/docker-entrypoint-initdb.d:ro
  mongo:
    image: mongo:9.0
    volumes: [ mongodata:/data/db ]
  redis:
    image: redis:7.4-alpine
    command: ["redis-server", "--appendonly", "yes"]
```

```
    volumes: [ redisdata:/data ]
volumes:
  pgdata:
  mongodata:
  redisdata:
```

D.2 Penambahan Integration Test Mandiri dan Uji Mutasi (Tugas Mandiri)

Sesuai ketentuan Tugas Mandiri D.2, telah ditambahkan dua integration test baru pada tests/integration/enroll_api_test.go yang mencakup alur Swap saat kelas tujuan penuh dan alur Drop saat berhasil keluar kelas, lengkap dengan perancangan dan pembuktian uji mutasinya.

1. Integration Test POST /enrollments/swap: Kelas Tujuan Sudah Penuh (TestSwapEndpoint_TargetCourseFull)

Skenario Uji: Mahasiswa A terdaftar di Kelas Asal (kuota 1, terisi 1). Kelas Tujuan juga telah terisi penuh oleh mahasiswa lain (kuota 1, terisi 1). Mahasiswa A mengirim request POST /api/v1/enrollments/swap untuk pindah ke Kelas Tujuan.
Invarian yang Divalidasi:
- Response API harus 409 Conflict dengan code COURSE_FULL.
- Mahasiswa A TETAP terdaftar di Kelas Asal (rollback pendaftaran).
- Mahasiswa A TIDAK terdaftar di Kelas Tujuan.
- enrolled_count Kelas Asal tetap 1 (tidak berkurang).
- enrolled_count Kelas Tujuan tetap 1 (tidak bertambah).
- Tidak ada event enrollment.swapped yang tertulis di tabel outbox.

```
func TestSwapEndpoint_TargetCourseFull(t *testing.T) {
    env := newIntegrationEnv(t)
    studentID := seedUser(t, env.pool, "swap_user", "swap@test.local", "user")
    token, _ := env.jwt.GenerateAccessToken(model.AuthUser{UserID: studentID, Username:
"swap_user", Role: "user"})

    fromID := seedCourse(t, env.pool, "SWAP101", 3, 1)
    toID := seedCourse(t, env.pool, "SWAP102", 3, 1)

    // Mahasiswa terdaftar di kelas asal
    seedEnrollment(t, env.pool, fromID, studentID)
    _, _ = env.pool.Exec(context.Background(), "UPDATE courses SET enrolled_count = 1 WHERE id =
$1", fromID)

    // Kelas tujuan diisi penuh oleh user lain
    otherID := seedUser(t, env.pool, "other_user", "other@test.local", "user")
    seedEnrollment(t, env.pool, toID, otherID)
    _, _ = env.pool.Exec(context.Background(), "UPDATE courses SET enrolled_count = 1 WHERE id =
$1", toID)

    reqBody, _ := json.Marshal(model.SwapRequest{FromCourseID: fromID, ToCourseID: toID})
    req := httptest.NewRequest(http.MethodPost, "/api/v1/enrollments/swap",
bytes.NewReader(reqBody))
    req.Header.Set("Content-Type", "application/json")
    req.Header.Set("Authorization", "Bearer "+token)

    resp, err := env.app.Test(req)
    if err != nil { t.Fatalf("request gagal: %v", err) }
    defer resp.Body.Close()

    // Assertions HTTP 409 COURSE_FULL, status kelas, dan outbox
    // (Seluruh kondisi terpenuhi 100%)
}
```

Rancangan Uji Mutasi Swap:
- Mutasi pada app/service/course_service.go (fungsi Swap): Mengabaikan error kuota tujuan: joined, _ = repo.IncrementEnrolled(ctx, req.ToCourseID).
- Bukti Kegagalan Test saat Dimutasi (Test Merah):

```
PS C:\Kuliah\Backend\Modul-12> go test -tags=integration -count=1 -vet=off -p 1 -buildvcs=false -
run TestSwapEndpoint_TargetCourseFull ./tests/integration/
--- FAIL: TestSwapEndpoint_TargetCourseFull (0.04s)
    enroll_api_test.go:251: swap kelas penuh: dapat status 200 code , seharusnya 409 COURSE_FULL
    enroll_api_test.go:258: mahasiswa harus tetap terdaftar di kelas asal, dapat count = 0
    enroll_api_test.go:265: mahasiswa tidak boleh terdaftar di kelas tujuan, dapat count = 1
    enroll_api_test.go:272: enrolled_count kelas asal berubah dari 1 menjadi 0
    enroll_api_test.go:282: tidak boleh ada event enrollment.swapped di outbox saat swap gagal,
dapat 1
FAIL
FAIL    latihan-fiber/tests/integration   0.347s
```

Hasil mutasi membuktikan bahwa test ini memvalidasi seluruh atomisitas transaksi, kuota, dan outbox.

2. Integration Test DELETE /courses/:id/enroll: Berhasil Drop Kelas (TestDropEndpoint_Success)

Skenario Uji: Mahasiswa terdaftar di Kelas Drop (kuota 5, terisi 1). Mahasiswa mengirimkan request DELETE /api/v1/courses/{id}/enroll.
Invarian yang Divalidasi:
- Response HTTP harus 204 No Content.
- Baris pendaftaran pada tabel enrollments terhapus (count = 0).
- enrolled_count kelas berkurang tepat satu (menjadi 0).
- Tabel outbox berisi tepat satu event enrollment.dropped dengan payload detail kelas.

```
func TestDropEndpoint_Success(t *testing.T) {
    env := newIntegrationEnv(t)
    studentID := seedUser(t, env.pool, "drop_user", "drop@test.local", "user")
    token, _ := env.jwt.GenerateAccessToken(model.AuthUser{UserID: studentID, Username:
"drop_user", Role: "user"})

    courseID := seedCourse(t, env.pool, "DROP101", 3, 5)
    seedEnrollment(t, env.pool, courseID, studentID)
    _, _ = env.pool.Exec(context.Background(), "UPDATE courses SET enrolled_count = 1 WHERE id =
$1", courseID)

    req := httptest.NewRequest(http.MethodDelete, fmt.Sprintf("/api/v1/courses/%d/enroll",
courseID), nil)
    req.Header.Set("Authorization", "Bearer "+token)

    resp, err := env.app.Test(req)
    if err != nil { t.Fatalf("request gagal: %v", err) }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusNoContent {
        t.Fatalf("drop kelas: status = %d, seharusnya 204", resp.StatusCode)
    }

    // Assert baris enrollment hilang, enrolled_count berkurang, outbox event tercatat
}
```

Rancangan Uji Mutasi Drop:
- Mutasi pada app/service/course_service.go (fungsi Drop): Mengganti pemanggilan DecrementEnrolled

dengan FindByID (lupa mengurangi kuota saat mahasiswa keluar kelas).
- Bukti Kegagalan Test saat Dimutasi (Test Merah):

```
PS C:\Kuliah\Backend\Modul-12> go test -tags=integration -count=1 -vet=off -p 1 -buildvcs=false -
run TestDropEndpoint_Success ./tests/integration/
--- FAIL: TestDropEndpoint_Success (0.03s)
    enroll_api_test.go:317: enrolled_count seharusnya 0, dapat 1
FAIL
FAIL    latihan-fiber/tests/integration   0.421s
```

Hasil mutasi membuktikan bahwa test ini memverifikasi penurunan kuota fisik di database secara presisi.

D.3 Kelengkapan Dokumentasi API OpenAPI / Swagger (Tugas Mandiri)

Sesuai ketentuan Tugas Mandiri D.3, seluruh endpoint yang belum memiliki anotasi telah dilengkapi dokumentasi Swagger-nya, dan seluruh response yang sebelumnya memakai tipe tidak terstruktur (fiber.Map) telah diberikan model bertipe khusus:
1. POST /auth/logout — Revokasi token refresh pengguna.
2. GET /auth/me — Mengambil data profil pengguna yang sedang login.
3. GET /users — Daftar seluruh akun pengguna dengan pagination dan filter role (khusus admin/staff).
4. GET /courses/{id}/reviews/summary — Agregasi statistik rating dan sebaran bintang ulasan mata kuliah.
5. GET /reviews/{id} — Mengambil dokumen detail satu ulasan dari MongoDB.
6. Endpoint User Lainnya: POST /users, GET /users/{id}, PUT /users/{id}, PATCH /users/{id}, PATCH /users/{id}/role, DELETE /users/{id}.
7. Endpoint Review Lainnya: PATCH /reviews/{id}, DELETE /reviews/{id}, POST /reviews/{id}/replies, POST /reviews/{id}/helpful.

Rangkuman Struktur docs/swagger.json:
- Total Paths: 20 path endpoints.
- Tags: 5 kelompok utama (auth, courses, reviews, users, notifications, admin).
- Definisi Model Tipe Baru: model.MyCourses, model.ReviewPage, model.ReviewPageMeta, model.ReviewSummary, model.TagCount, model.AssignRoleRequest.

```
Daftar 20 Endpoint yang Terdaftar di Swagger:
[auth]
- POST   /api/v1/auth/register : Pendaftaran akun baru
- POST   /api/v1/auth/login    : Autentikasi dan penerbitan token
- POST   /api/v1/auth/refresh  : Rotasi refresh token
- POST   /api/v1/auth/logout   : Pencabutan refresh token
- GET    /api/v1/auth/me       : Profil akun aktif
[courses]
- GET    /api/v1/courses             : Daftar seluruh kelas
- GET    /api/v1/courses/mine        : KRS mahasiswa aktif
- GET    /api/v1/courses/{id}        : Detail satu kelas
- POST   /api/v1/courses/{id}/enroll : Pendaftaran KRS
- DELETE /api/v1/courses/{id}/enroll : Batal pendaftaran KRS (drop)
- POST   /api/v1/enrollments/swap    : Pindah kelas (swap)
[reviews]
- POST   /api/v1/courses/{id}/reviews         : Tulis ulasan kelas
- GET    /api/v1/courses/{id}/reviews         : Daftar ulasan kelas (cursor pagination)
- GET    /api/v1/courses/{id}/reviews/summary : Ringkasan rating ulasan
- GET    /api/v1/reviews/{id}                 : Detail satu ulasan
- PATCH  /api/v1/reviews/{id}                 : Update isi ulasan
- DELETE /api/v1/reviews/{id}                 : Hapus ulasan
- POST   /api/v1/reviews/{id}/replies         : Balas ulasan
- POST   /api/v1/reviews/{id}/helpful         : Beri apresiasi ulasan
```

```
[users]
- GET    /api/v1/users           : Daftar pengguna
- POST   /api/v1/users           : Tambah pengguna baru
- GET    /api/v1/users/{id}      : Detail pengguna
- PUT    /api/v1/users/{id}      : Perbarui seluruh data pengguna (replace)
- PATCH  /api/v1/users/{id}      : Perbarui sebagian data pengguna (partial update)
- PATCH  /api/v1/users/{id}/role : Ubah role pengguna
- DELETE /api/v1/users/{id}      : Hapus pengguna
[notifications & admin]
- GET    /api/v1/notifications/mine : Riwayat notifikasi mahasiswa
- GET    /api/v1/admin/queue        : Monitoring outbox dan antrean Redis
```

D.4 Analisis Singkat (Tugas Mandiri)

1. Hubungan Antara Code Coverage dan Uji Mutasi (Soal 1)

a. Mengapa Code Coverage Tidak Dapat Menggantikan Uji Mutasi:
Code coverage adalah metrik kuantitatif yang hanya mengukur apakah suatu baris kode atau instruksi dieksekusi selama pengetesan, tanpa pernah menilai apakah pengujian tersebut memverifikasi kebenaran hasil eksekusi tersebut (assertion validation). Sebagai contoh konkret pada fungsi WithinCreditLimit:
Sebelum skenario batas ditambahkan, coverage fungsi tersebut telah mencapai 100% hanya dengan skenario taken: 10, adding: 3. Ketika kode sengaja dimutasi dengan mengganti operator '<=' menjadi '<', seluruh baris fungsi tetap dieksekusi sehingga laporan coverage tetap berwarna hijau (100%), padahal logika bisnis batas 24 SKS telah rusak. Uji mutasi bertindak sebagai penilai kualitas test suite dengan menyuntikkan kerusakan logika buatan (mutant) dan menuntut test suite untuk menjadi merah (gagal). Test suite yang memiliki coverage tinggi namun membiarkan mutant tetap hidup (survived) adalah test suite yang semu.

b. Mengapa Uji Mutasi Tidak Dapat Menggantikan Code Coverage:
Uji mutasi bekerja dengan memodifikasi baris-baris kode yang sudah ada. Namun, jika terdapat cabang kode (branch) atau fungsi baru yang sama sekali belum pernah dipanggil oleh test runner (uncovered code), uji mutasi tidak dapat memberikan informasi yang bermakna karena mutasi pada baris tersebut tidak akan pernah memicu kegagalan test suite. Oleh karena itu, code coverage berfungsi sebagai peta awal untuk menemukan bagian sistem yang belum tersentuh tes, sedangkan uji mutasi berfungsi sebagai tolok ukur kualitatif untuk memastikan ketajaman asersi pada kode yang sudah terjangkau.

2. Analisis Perilaku Fake MongoDB vs MongoDB Sungguhan (Soal 2)

a. Perilaku MongoDB Sungguhan yang TIDAK Ditiru oleh Fake:
Fake pada Langkah 2 (fakeReviewRepo) meniru unique index (satu mahasiswa satu ulasan per kelas) dengan melakukan perulangan for di memori Go secara sinkron (single-threaded). Perilaku sistemik MongoDB sungguhan yang TIDAK ditiru antara lain:
1. Write Conflict dan E11000 Duplicate Key Error pada Konkurensi Tinggi: Di database MongoDB nyata dengan storage engine WiredTiger, dua request pembuatan ulasan yang masuk secara bersamaan akan ditangani oleh thread terpisah. Mesin MongoDB mendeteksi pelanggaran unique compound index {course_id: 1, user_id: 1} pada level b-tree index dan melempar error driver kode 11000. Fake memory Go yang dilindungi mutex (sync.Mutex) hanya mensimulasikan eksekusi serial, bukan perilaku konkurensi database nyata.
2. Pipeline Agregasi dan Indeks Teks: Fake tidak meniru operator agregasi ($group, $avg, $push) atau filter array pada MongoDB sungguhan.

b. Jenis Test yang Seharusnya Memeriksanya:
Perilaku ini seharusnya diuji menggunakan Integration Test (atau Database Contract Test) yang terhubung langsung ke kontainer MongoDB terisolasi (misalnya menggunakan testcontainers-go atau instans database test compose). Pengujian integrasi ini mengeksekusi request konkurensi untuk memastikan driver go.mongodb.org/mongo-driver menangani error E11000 secara tepat dan mengonversinya menjadi HTTP 409 Conflict.

3. Strategi Menjalankan Database Migration pada Docker Compose (Soal 3)

a. Perbandingan Strategi Migration:
- Docker Compose (docker-entrypoint-initdb.d): Script SQL dieksekusi secara otomatis oleh image postgres resmi, HANYA saat volume data (pgdata) masih kosong. Kelebihannya adalah isolasi proses penuh dan jaminan skema telah selesai dibuat sebelum aplikasi mulai menyala. Namun, kekurangannya adalah perubahan skema baru (seperti file migrasi 009) tidak otomatis berjalan pada volume yang sudah memiliki data.
- In-App Migration (seperti TestMain): Aplikasi membaca dan menjalankan file SQL setiap kali proses menyala. Kelebihannya adalah skema selalu mutakhir secara otomatis tanpa campur tangan administrator.

b. Kelebihan dan Risiko Bila API Dijalankan Tiga Replika Sekaligus:
- Kelebihan In-App: Pengembang tidak perlu repot menjalankan script migrasi manual saat deploy.
- Risiko Sangat Fatal: Terjadi Race Condition antar Replika. Ketika tiga kontainer API menyala serentak, ketiganya akan mengeksekusi script DDL (CREATE TABLE, ALTER TABLE, ADD CONSTRAINT) secara paralel. Hal ini memicu:
  1. Error SQLSTATE 42P07: relation already exists saat dua replika mencoba membuat index/constraint yang sama secara simultan.
  2. Database Deadlock: Eksekusi DDL memegang exclusive lock pada catalog database PostgreSQL, menyebabkan replika saling mengunci dan gagal startup (crash loop backoff).
- Solusi Terbaik di Production: Jika menggunakan in-app migration, aplikasi WAJIB menggunakan Distributed Advisory Lock (misalnya SELECT pg_advisory_lock(hash_key)) sehingga hanya satu replika yang memegang hak migrasi, atau memisahkan proses migrasi sebagai Kubernetes InitContainer / Docker One-off Job yang selesai sebelum replika API dinyalakan.

4. Analisis Risiko Keamanan Swagger UI Terbuka di Lingkungan Kampus (Soal 4)

Bila docker compose up dijalankan di server kampus dengan SWAGGER_ENABLED=true dan port 8080 terbuka ke internet publik, terdapat dua risiko konkret beserta perbaikannya:

a. Risiko 1: Information Disclosure dan Cetak Biru Penyerangan (Reconnaissance Mapping)
- Dampak Konkret: Dokumentasi OpenAPI mengekspos seluruh topologi sistem secara transparan: daftar lengkap 20 endpoint, nama parameter query/path, tipe data payload JSON, header autentikasi yang dibutuhkan, serta kode error internal. Penyerang tidak perlu menebak-nebak (blind fuzzing); mereka dapat langsung melihat endpoint administratif seperti GET /api/v1/admin/queue atau struktur PATCH /api/v1/users/{id}/role untuk merancang serangan privilege escalation.
- Solusi Perbaikan: Nonaktifkan Swagger di production dengan menyetel SWAGGER_ENABLED=false pada environment server kampus. Jika dokumentasi tetap diperlukan oleh tim internal, batasi akses URL /swagger/* hanya untuk IP intranet kampus (IP Whitelisting) atau lewat koneksi VPN kampus di level Reverse Proxy (Nginx/Traefik).

b. Risiko 2: Interactive Exploit Surface dan Serangan Brute-Force / DoS melalui Browser
- Dampak Konkret: Swagger UI menyediakan fitur interaktif 'Try it out' yang memudahkan penyerang

awam (script kiddies) untuk mengirimkan request HTTP langsung dari browser tanpa memerlukan Postman/curl. Penyerang dapat memanfaatkan endpoint POST /api/v1/auth/login untuk melakukan credential stuffing atau membombardir endpoint berat (seperti agregasi ulasan atau sinkronisasi queue) sehingga membebani CPU dan memori server kampus (Denial of Service).
- Solusi Perbaikan: Lindungi rute Swagger dengan HTTP Basic Authentication di level web server, terapkan Rate Limiting ketat pada Reverse Proxy, dan pastikan container API hanya mendengarkan pada antarmuka lokal (127.0.0.1:8080) di balik gateway resmi.